

MSE 413 Lab 3 Report

Clustering and Dimensionality Reduction (PCA)

Name: Tom Dang

Student ID: 301548582

Simple K-Means and K-Medoids Clustering

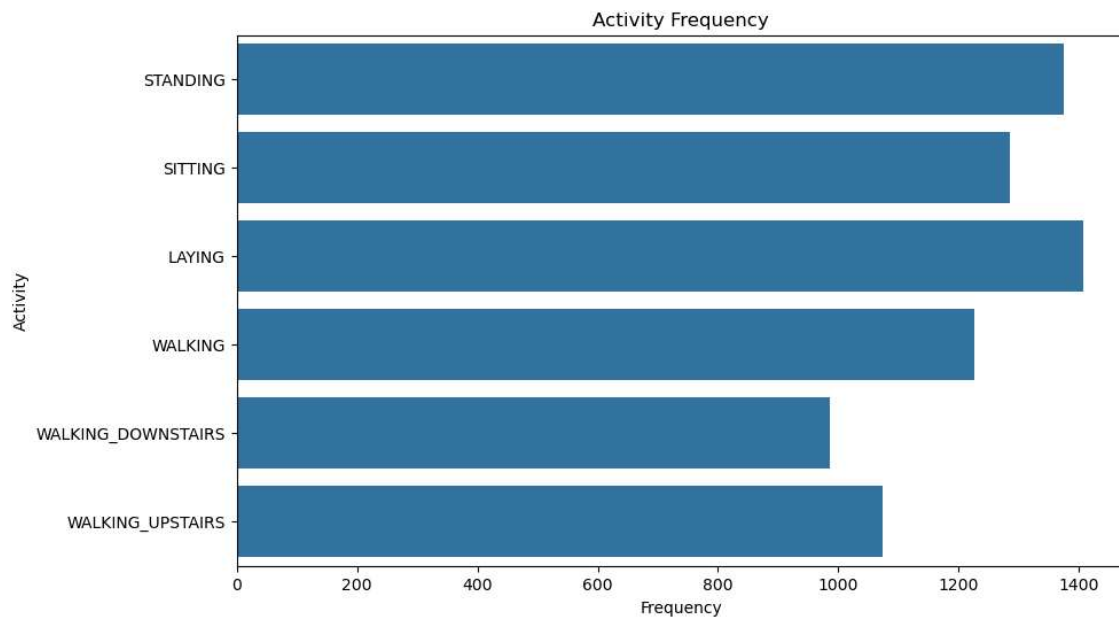


Figure 1: Sample sizes for each predicted outcome

The number of samples representing each activity is fairly evenly distributed for this dataset. The samples of walking downstairs are the lowest in this dataset. The samples for laying is the most abundant.

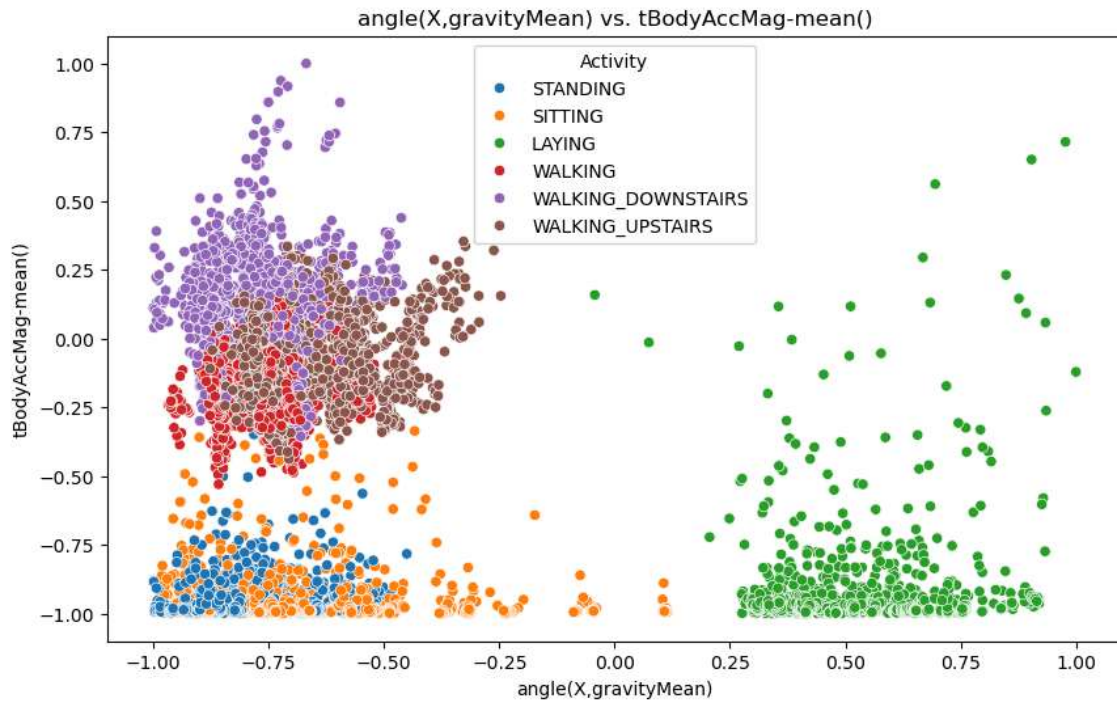


Figure 2: Visualization of data samples for different categories

- Laying is a distinctive activity in this dataset.
- Walking, walking upstairs or downstairs is harder to distinct with angular acceleration and gravity mean, but is distinct from other activities meantioned in the graph
- Standing and sitting can not be distinguish apart by be told different from all kinds of walking activities and laying

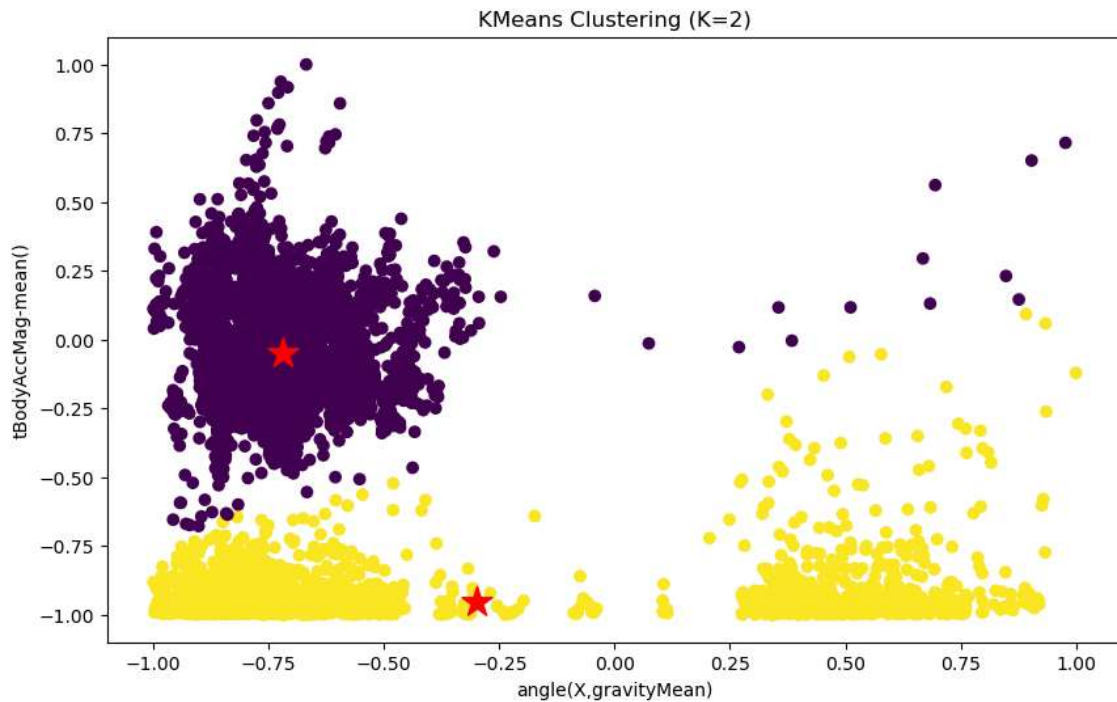


Figure 3: K-Means Clustering (K=2)

Table 1: Centroids of clusters of K-Means Clustering (K=2)

Centroid #	X	Y
1	-0.29998306	-0.95533349
2	-0.71862997	-0.04665947

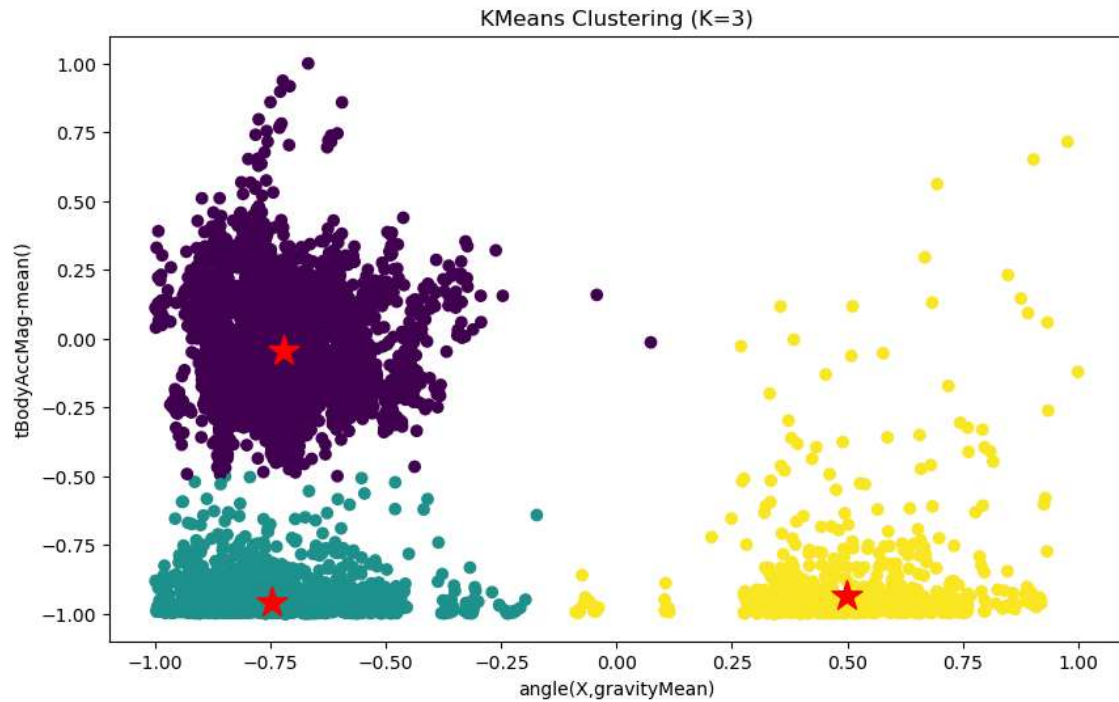


Figure 4: K-Means Clustering (K=3)

Table 2: Centroids of clusters of K-Means Clustering (K=3)

Centroid #	X	Y
1	-0.72245738	-0.04471589
2	-0.74802428	-0.95829744
3	0.49919599	-0.93635537

The number of clusters divided the zone in the dataset differently, depending on the number of clusters to divide into. The regions are divided so that the centroids each have an equal area of coverage. As the number of centroids increases, the distance between data points decreases. A K that is too small leads to distinct data points being in the same group. A too large K leads to unnecessary fragmentation in classification. K values in this model seem to be sensitive to outliers because it tries to minimize the squared error between centroids and data points.

In the 2 clusters, the model can recognize the difference between active (all sorts of walking) and inactive (sitting, standing, lying). In the 3-cluster diagram, the model can tell the difference between

walking and (sitting, standing) and laying. Compared to the actual result, the outliers for sitting and lying are not clustered correctly.

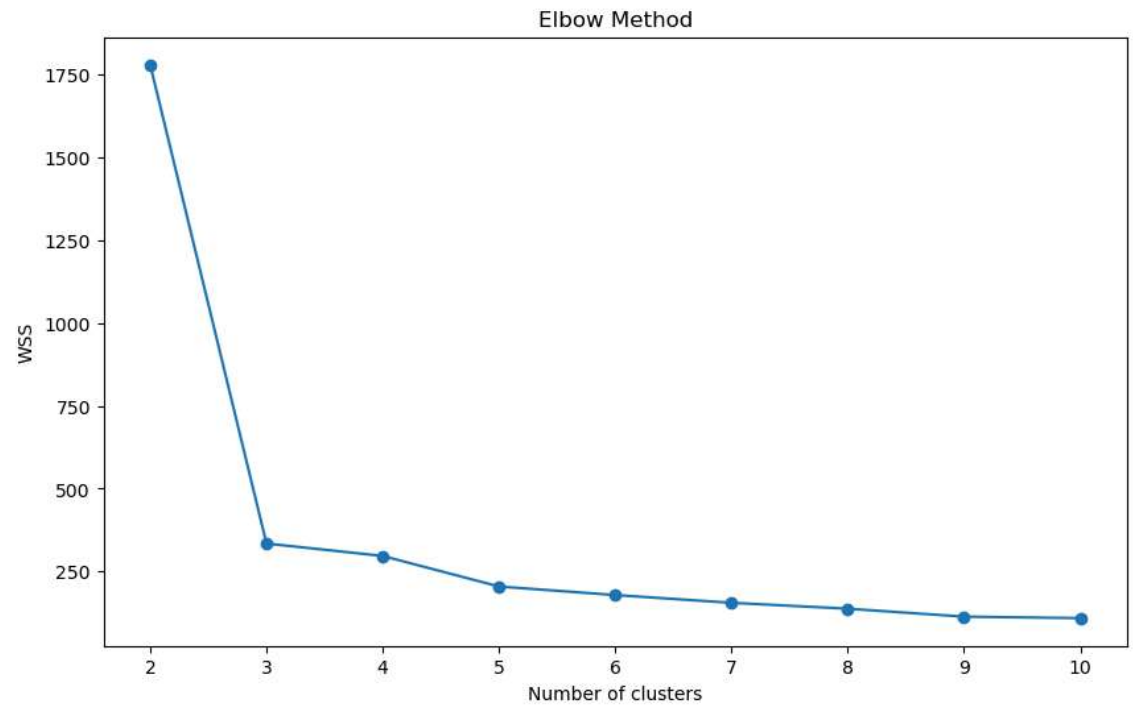


Figure 5: Evaluation of the optimal neighbors for K-Means clustering using elbow method

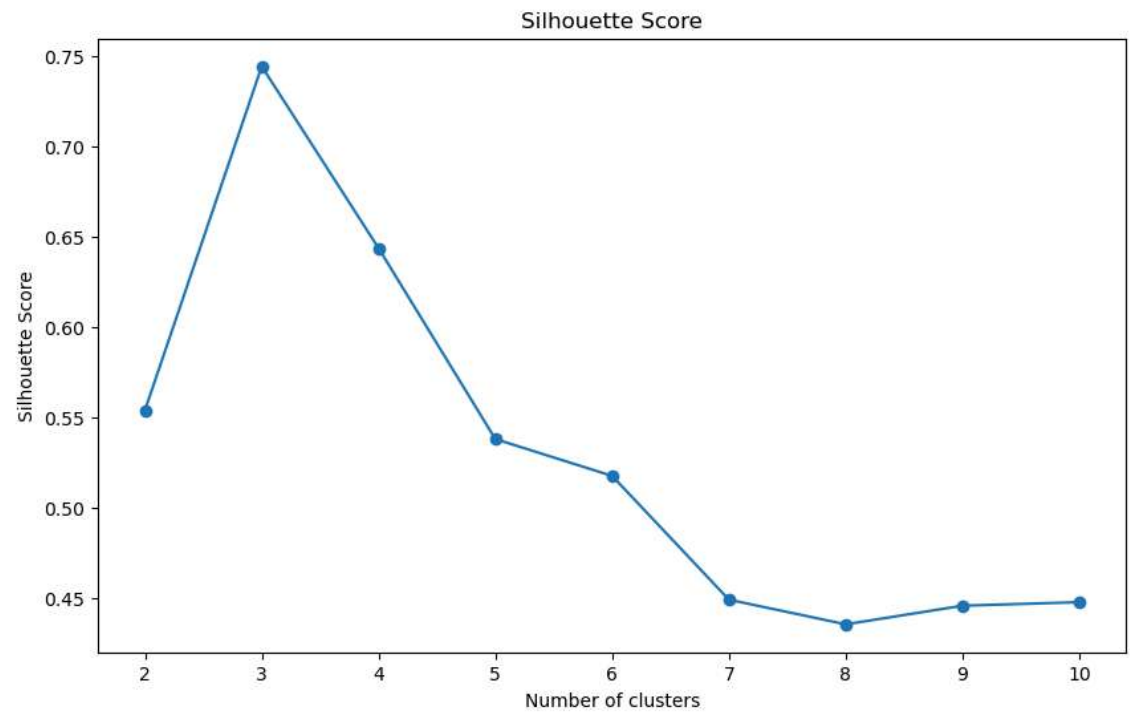


Figure 6: Evaluation of the optimal neighbors for K-Means clustering using highest silhouette score

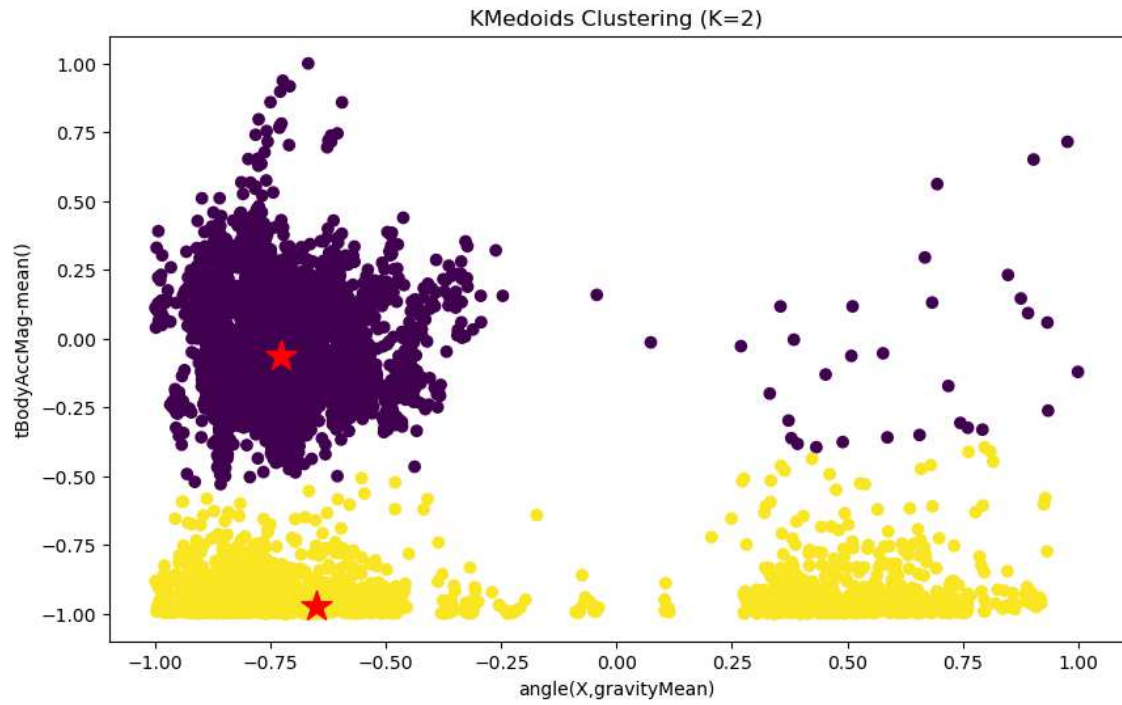


Figure 7: K-Medoids clustering (K=2)

Table 3: Centroids of K-Medoids (K=2)

Centroid #	X	Y
1	-0.72693856	-0.06464965
2	-0.64988656	-0.97293097

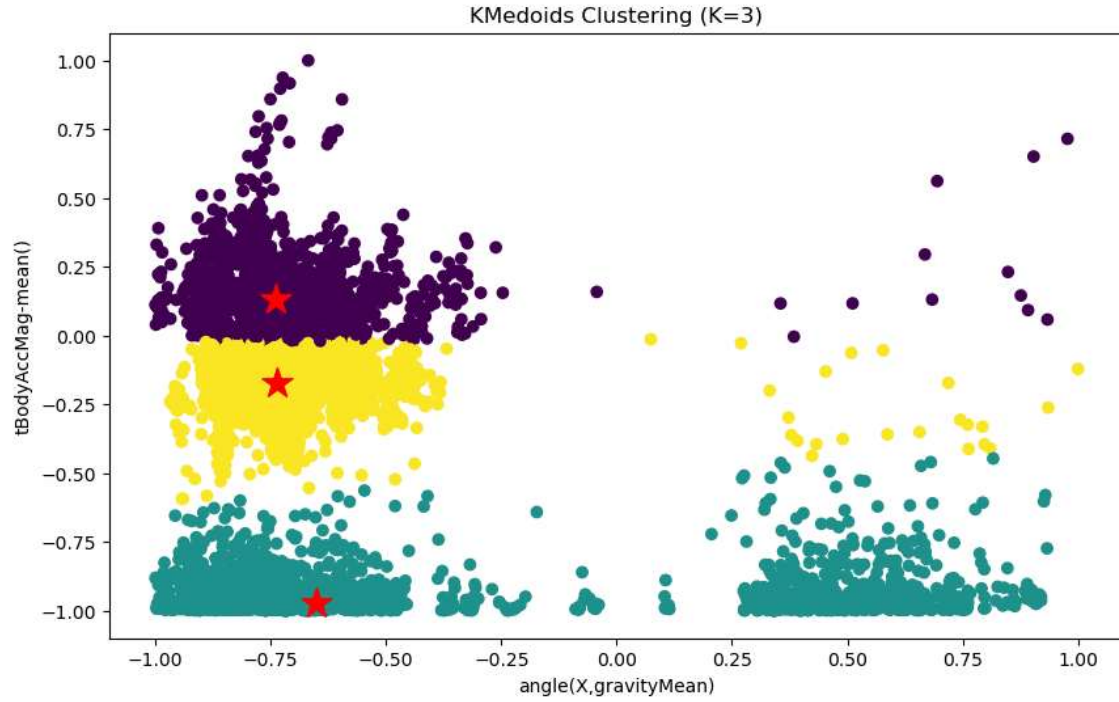


Figure 8: K-Medoids clustering (K=3)

Table 4: Centroids of K-Medoids clustering (K=3)

Centroid #	X	Y
1	-0.73957584	0.13018951
2	-0.64988656	-0.97293097
3	-0.7360151	-0.1730461

The K-Medoids model determines by merging the data points into larger centroids, which is less sensitive to outliers. The effect is similar to the K-Means model. A small k leads to distinct data points treated the same category. A large k leads to fragmentation.

The K-Metroid can distinguish between stationary (sitting, standing, laying) and moving (all sorts of walking) activities. In cluster=3, the model does not differentiate between sitting, standing and lying, but vaguely differentiate between different types of walking (because there are many blends between the 3 different types of walking).

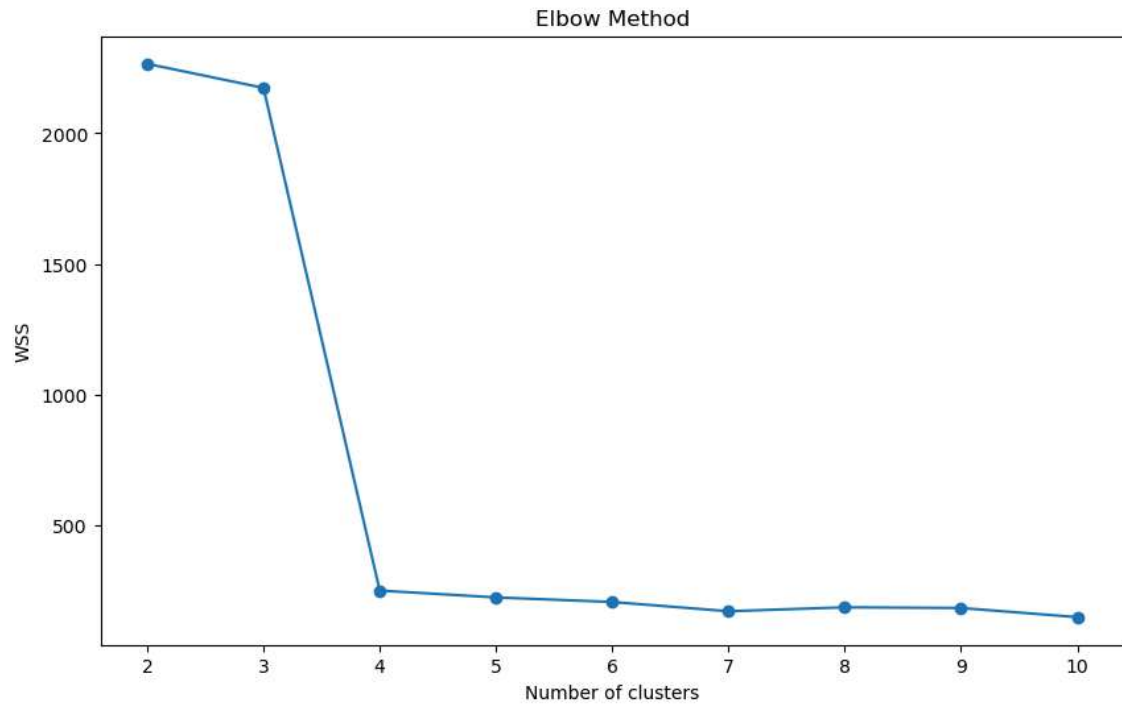


Figure 9: Evaluation on optimal neighbors for K-Medoids clustering using elbow method

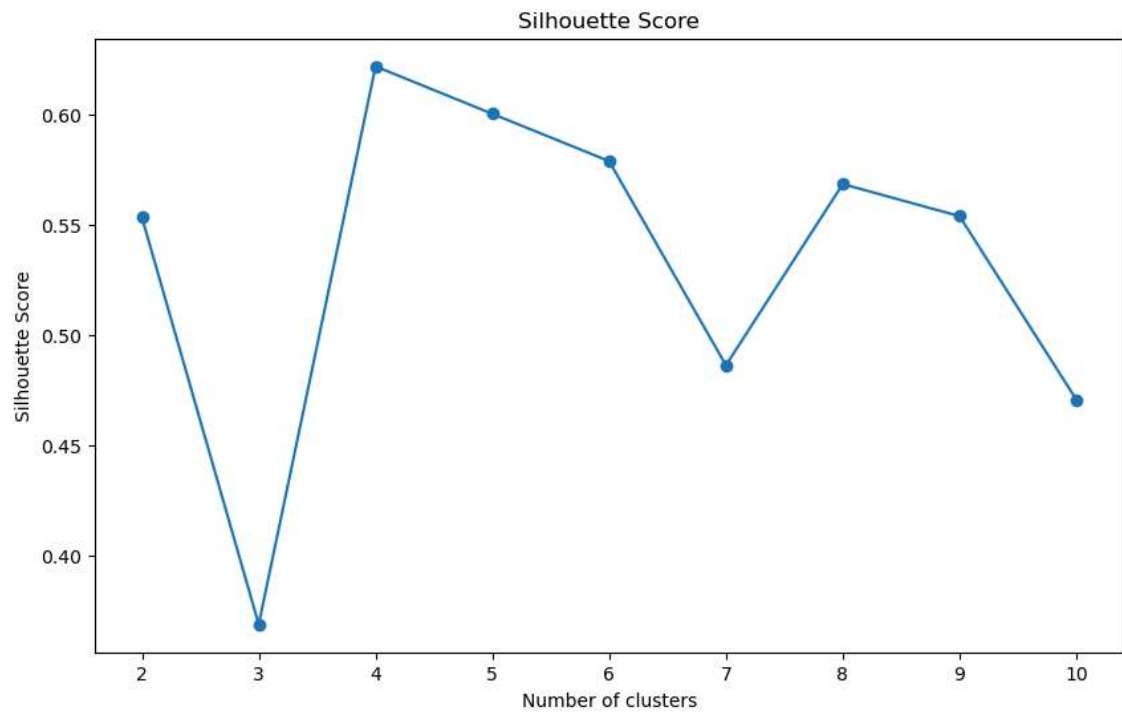


Figure 10: Evaluation on optimal neighbors for K-Medoids clustering using silhouette score

Inertia and Silhouette Score

The inertia (sum of squared distance of samples to their closest cluster center) decreases as the number of clusters increases. This is a reasonable observation because there are more centers to choose for the closest center to a specific data point. The data is fragmental as the cluster increases and no longer reflects the actual data. The same behavior is exhibited on the K-Medoids clustering model.

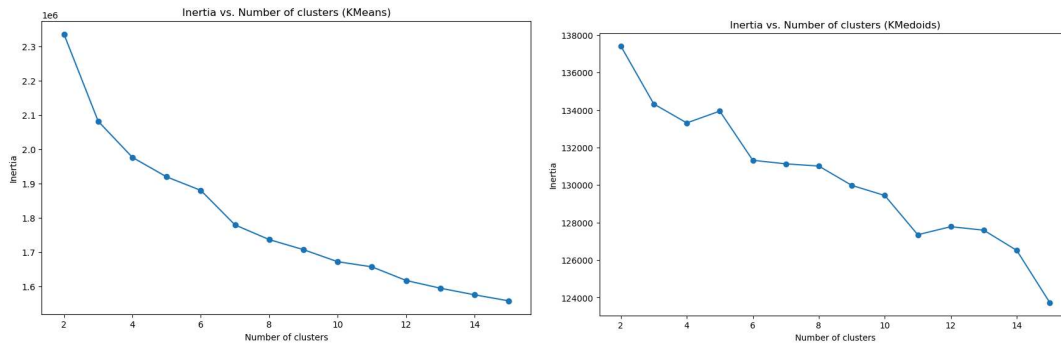


Figure 11: Comparison between Inertia and number of clusters on K-Mean model

Based on the actual data of which data points and their representation of activities, the cluster of 4 is the optimal since it gives the closest answer to the actual data representation. Walking data is messy, blending into each other a lot, but it can be generalized into just walking. The sitting and standing data is accurate based on the "KMeans Clustering (K=4)".

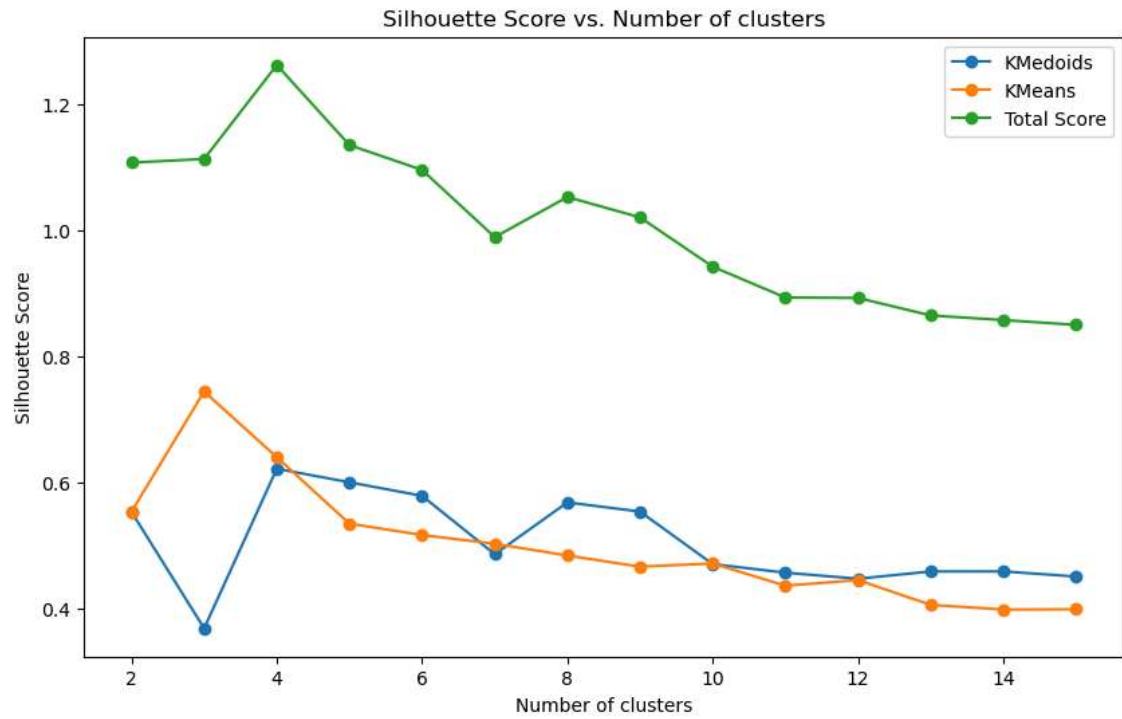


Figure 12: Silhouette scores on both models for final optimal neighbors for both models

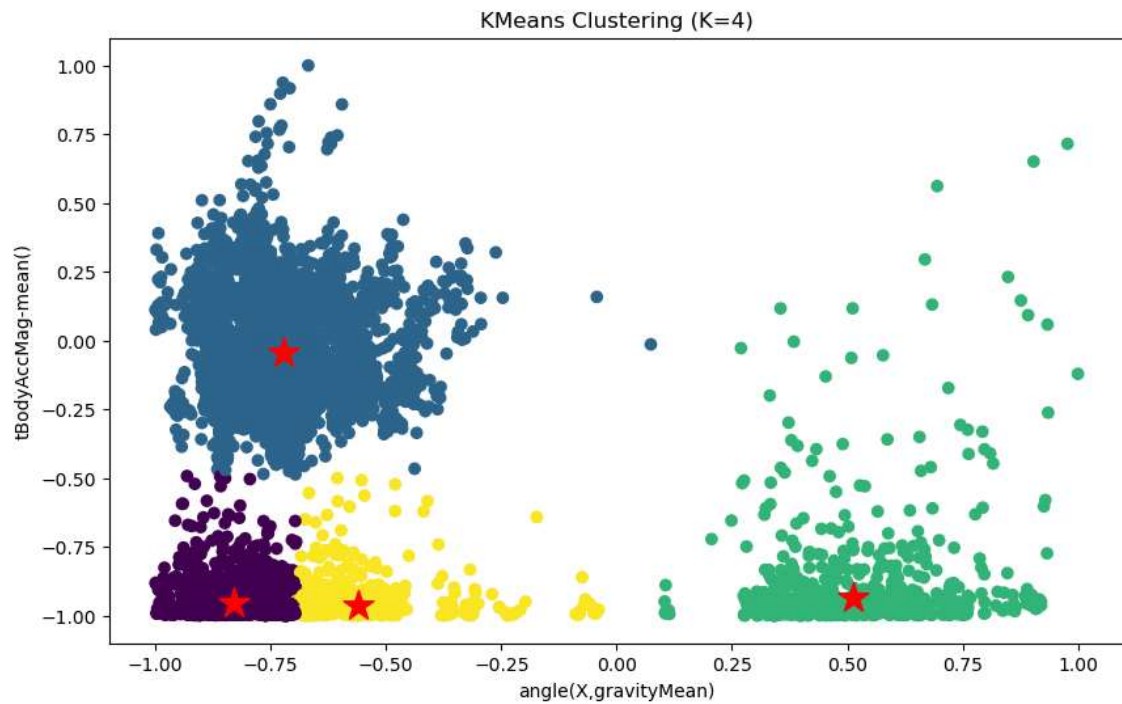


Figure 13: K-Means clustering (K=4)

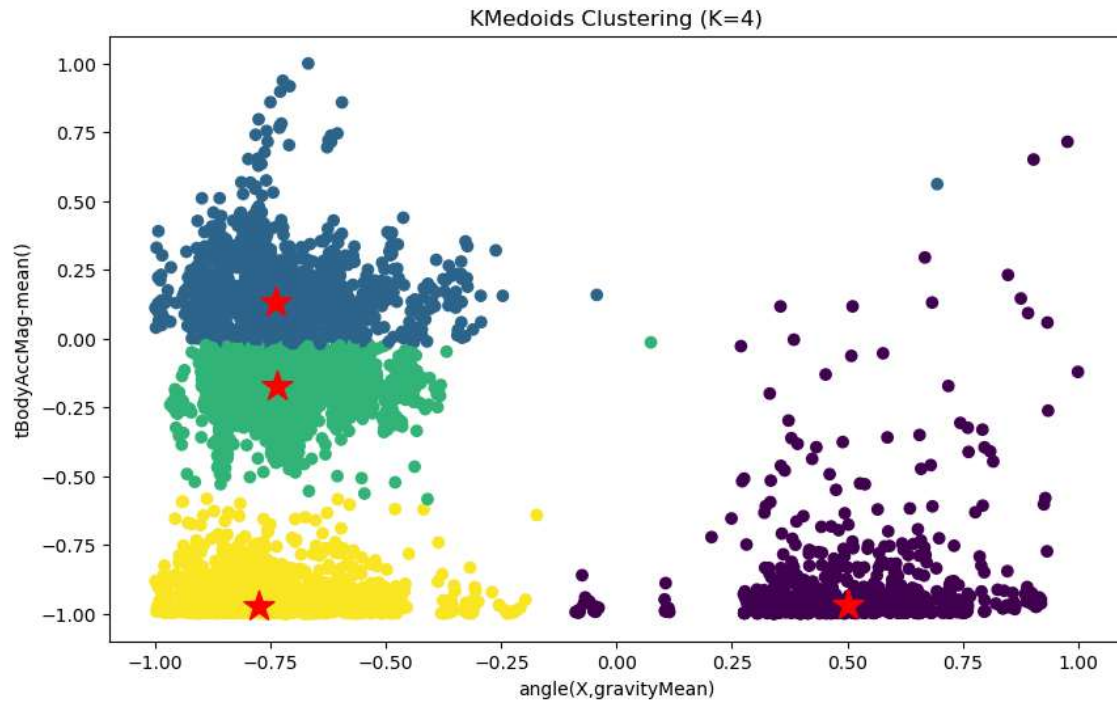


Figure 14: K-Medoids clustering (K=4)

Given the benchmark for silhouette score, the highest score is the most optimal cluster for each model. The K-means has the highest score at K=3 with about 0.75 score, which means the data is well-separated (walking, station, laying). The K-medoid has the highest score when K=4, which has a separation in sitting, standing, lying, and walking. Sitting and standing have some overlap, so the score drops below 0.70. Based on the total score, the individual model's score is about 0.60. The cluster of K=4 is optimal for both models, and it also implies there are data overlaps in the dataset, such as sitting and standing.

Silhouette score benchmark

- 0.70 to 1.00: Indicates a very good clustering configuration with well-separated clusters.
- 0.50 to 0.70: Suggests that the clustering is moderate; while some clusters are distinguished, there may still be some overlaps.
- 0.25 to 0.50: Implies poor clustering, suggesting that many items may be classified incorrectly or clusters may be overlapping excessively.
- Below 0.25: Indicates a very poor clustering situation, often suggesting that the number of clusters may need to be reevaluated or that a different algorithm might yield better results.

Source: [Silhouette Score Benchmark

Guide](<https://futuremachinelearning.org/understanding-silhouette-score-a-key-metric-for-clustering/>)

Semi-Supervised Learning

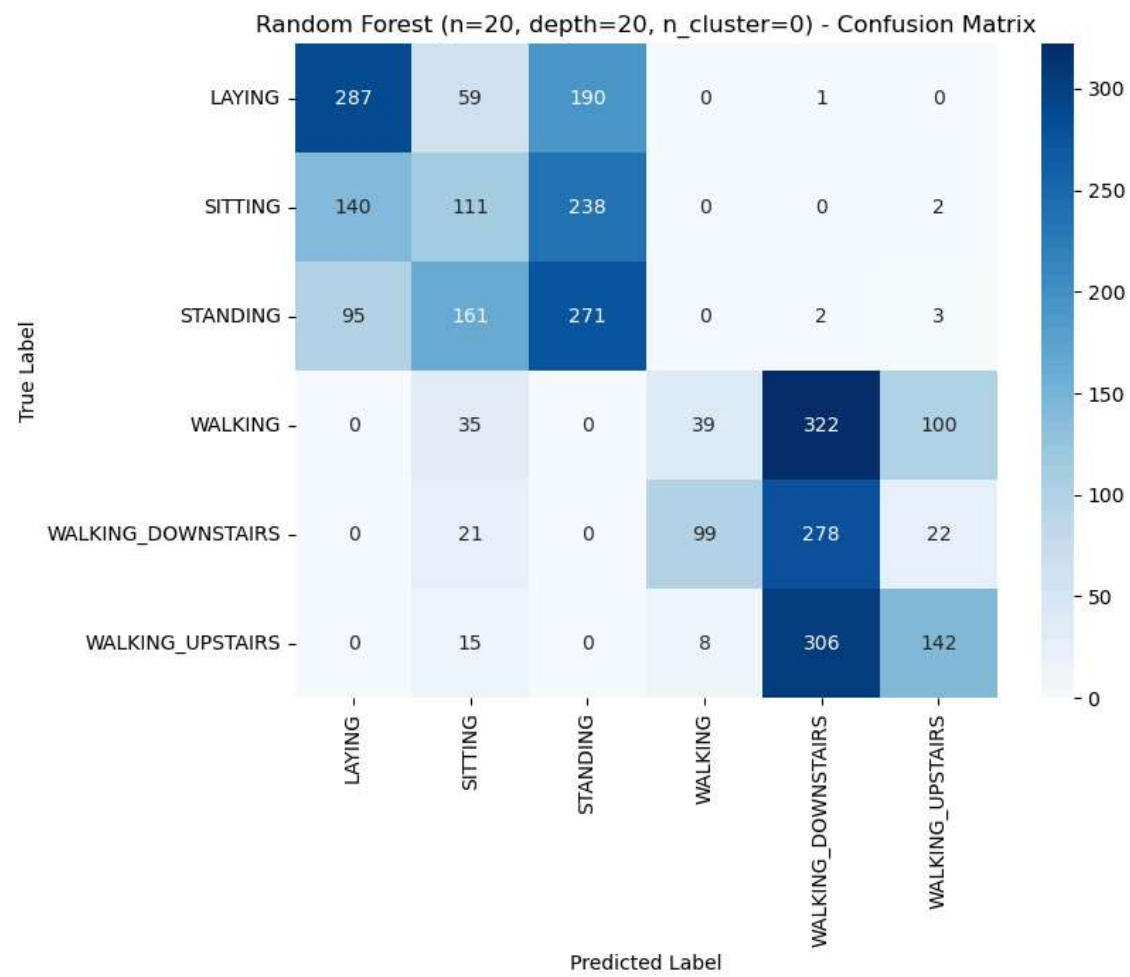


Figure 15: Confusion matrix when testing random forest model with minimal raw data

Table 5: Metrics for model evaluation

Accuracy	0.3828
Sensitivity	0.3853
Specificity	0.8770
F1 Score	0.3591

The model has a high specificity score but fails on other scores due to limited labelled data—in this case, six samples, one for each case. The model can distinguish between the machine's moving and stationary stages. Improvements can be made by increasing the number of possible samples or auto-labeling samples with high confidence.

Pseudo-code completion

```
''' python
import numpy as np
import pandas as pd
from pathlib import Path
from numpy import linalg
from sklearn.cluster import KMeans

Data_unlabeled = pd.read_csv(Path("/content/Human activity semi supervised.csv"))
Data_unlabeled2= Data_unlabeled.dropna()
Data_labeled=Data_unlabeled.copy()
Data_train= Data_unlabeled.drop(["subject","Activity"], axis=1)

n_clusters=9
kmeans = KMeans(n_clusters=n_clusters, random_state=0, n_init="auto").fit(Data_train)
kmeans_2=kmeans.labels_
centers = kmeans.cluster_centers_

Features= Data_unlabeled2.drop(["subject","Activity"], axis=1)
Feature=Features.to_numpy()

num_rows, num_cols = Feature.shape
dist=[None]*num_rows
for i in range(0,n_clusters):
    for j in range(0,len(num_cols)):
        dist[j] = np.linalg.norm(centers[i]-Feature[j])
    index_min = np.argmin(dist)
    for k in range(0,len(kmeans_2)):
        if kmeans_2[k]==i:
            Data_labeled.loc[k,"Activity"]=Data_unlabeled.loc[Features.index[index_min],"Activity"]
'''
```

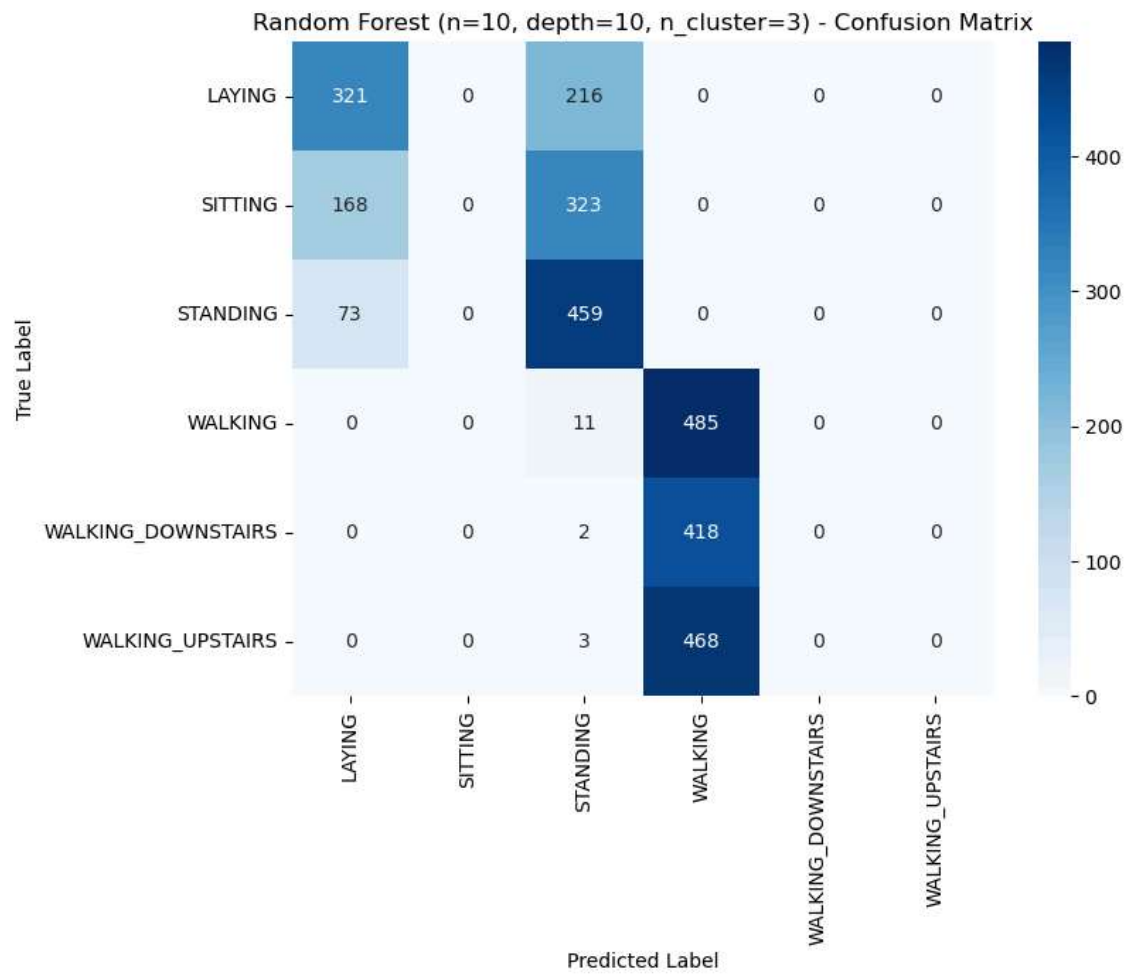


Figure 16: Semi-supervised training with K-Means ($K=3$) and random forest model

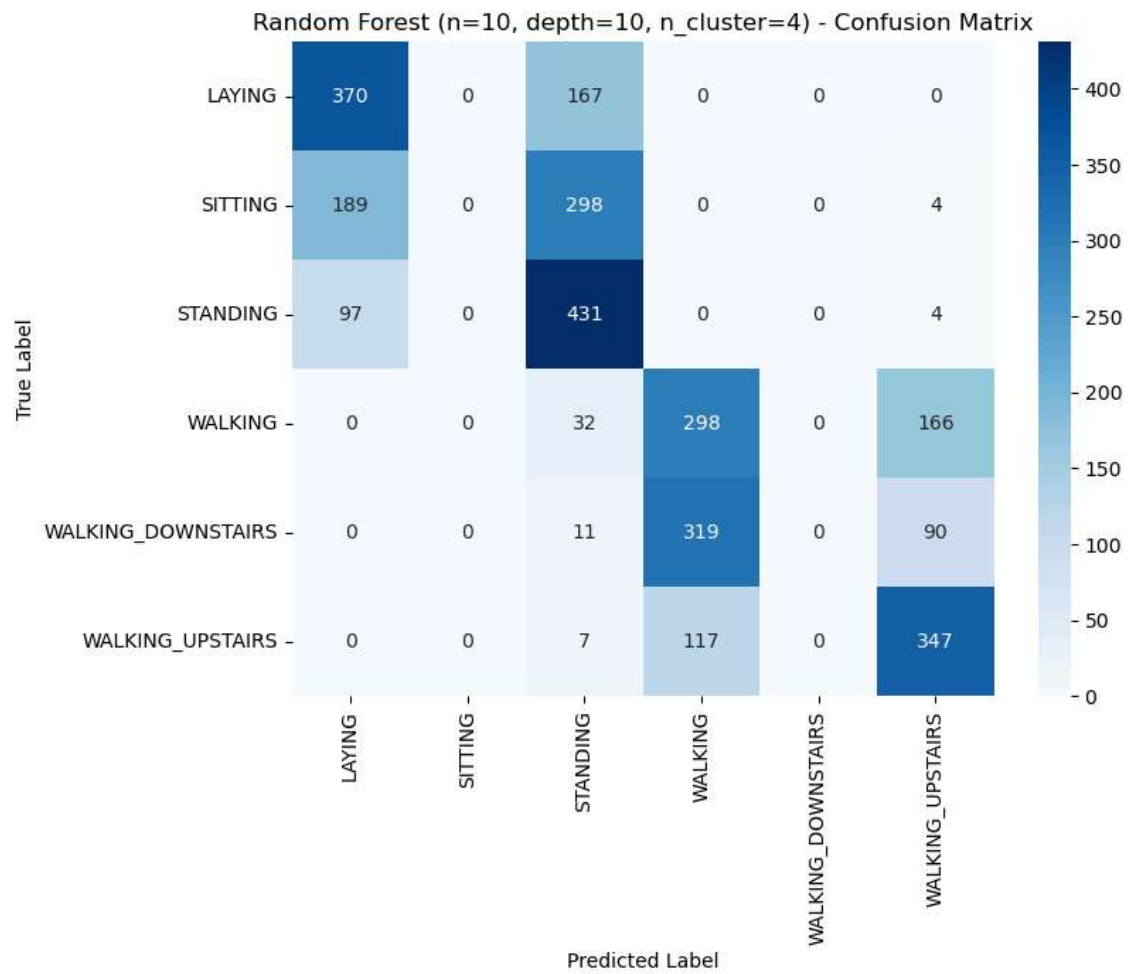


Figure 17: Semi-supervised training with K-Means ($K=4$) and random forest model

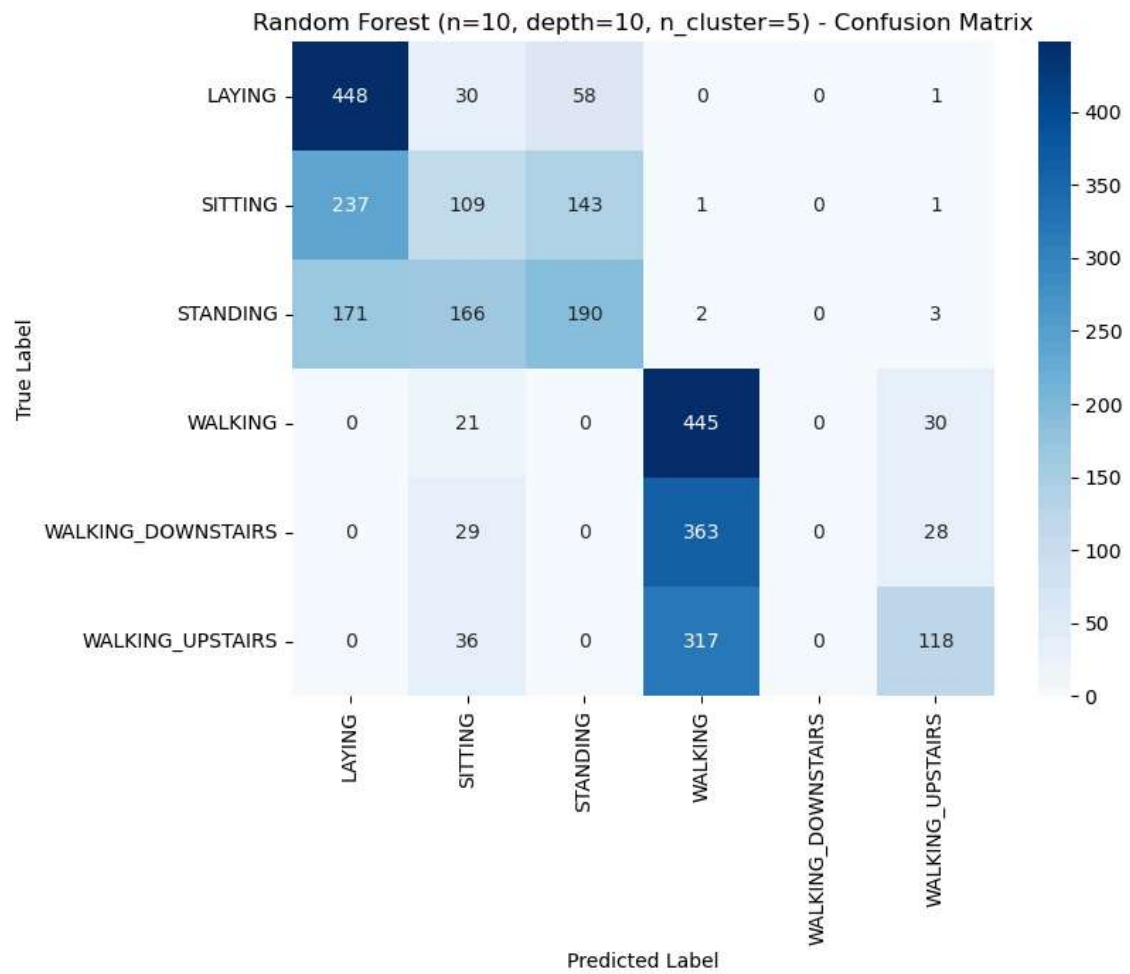


Figure 18: Semi-supervised training with K-Means ($K=5$) and random forest model

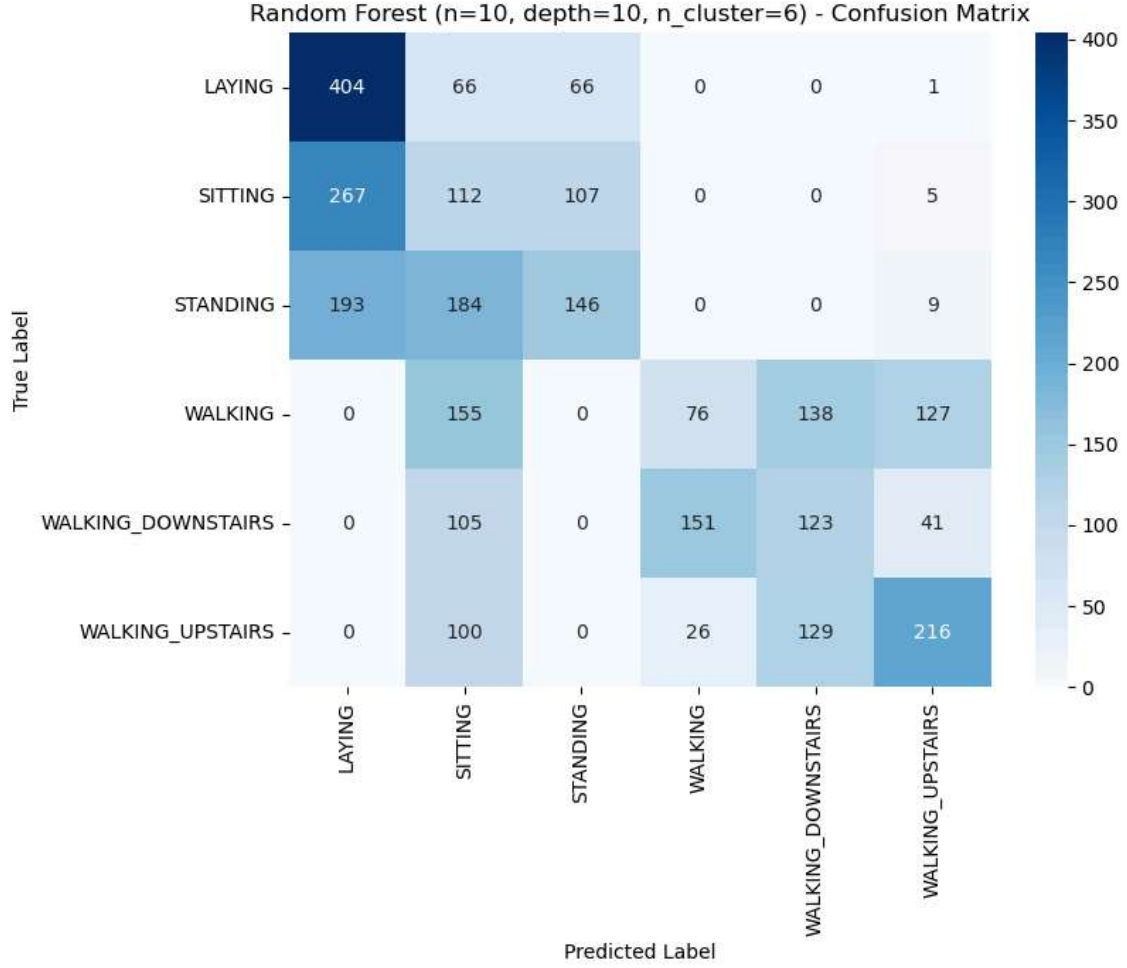


Figure 19: Semi-supervised training with K-Means (K=6) and random forest model

Table 6: Semi-supervised metrics for evaluation

Clusters	Accuracy	Sensitivity	Specificity	F1 Score
3	0.4293	0.4064	0.8848	0.2829
4	0.4907	0.4728	0.8973	0.3882
5	0.4445	0.4269	0.8881	0.3687
6	0.3655	0.3599	0.8729	0.3513

Since the samples are limited to 6 labeled samples, there are a maximum of 6 clusters to divide. Any number of clusters above 7 is impossible because the model relies on data points to mark boundaries between and among other data points. The smallest possible number of clusters is 2.

The accuracy is improved in the later model, and the other metrics in the same cluster of 6 are relatively similar. The best in this iteration is one with 4 clusters with the highest accuracy, sensitivity, specificity, and F1 score. Overall, the dataset is imbalanced, which leads to a poor model with extremely high specificity but low on others. 4 cluster labeling is the best it can do. This model can

differentiate between laying, station (standing and sitting), walking upstairs, and other walking (walking and walking downstairs).

It would be helpful to have more labeled samples before putting them into the K-mean model. In this case, the specificity is especially high, while other metrics are poor.

- The dataset is imbalanced, proven by the F1 score. The error source is the mislabel in the K-means model
- The model can only identify negative cases (high specificity). It struggles to generalize and correctly classify actual positive cases.
- Due to the nature of the dataset, it has overlapping data, which makes it difficult to classify different classes. Since there are 6 different unique activities, the best possible number of clusters is 6, but it can be more generalized into 3 because sitting and standing are stationary, and walking, walking upstairs, and walking downstairs are moving in general.

Possible solutions to solve the imbalanced dataset

- Tune the number of estimators and the depth of the random forest model
- Use different clustering methods besides K-mean
- Use other methods for semi-supervised
- Add a model or strategies to handle the imbalanced dataset
- Re-label the dataset to 3 activities only to improve the model's ability to generalize the information

Principal Component Analysis for Dimensionality Reduction

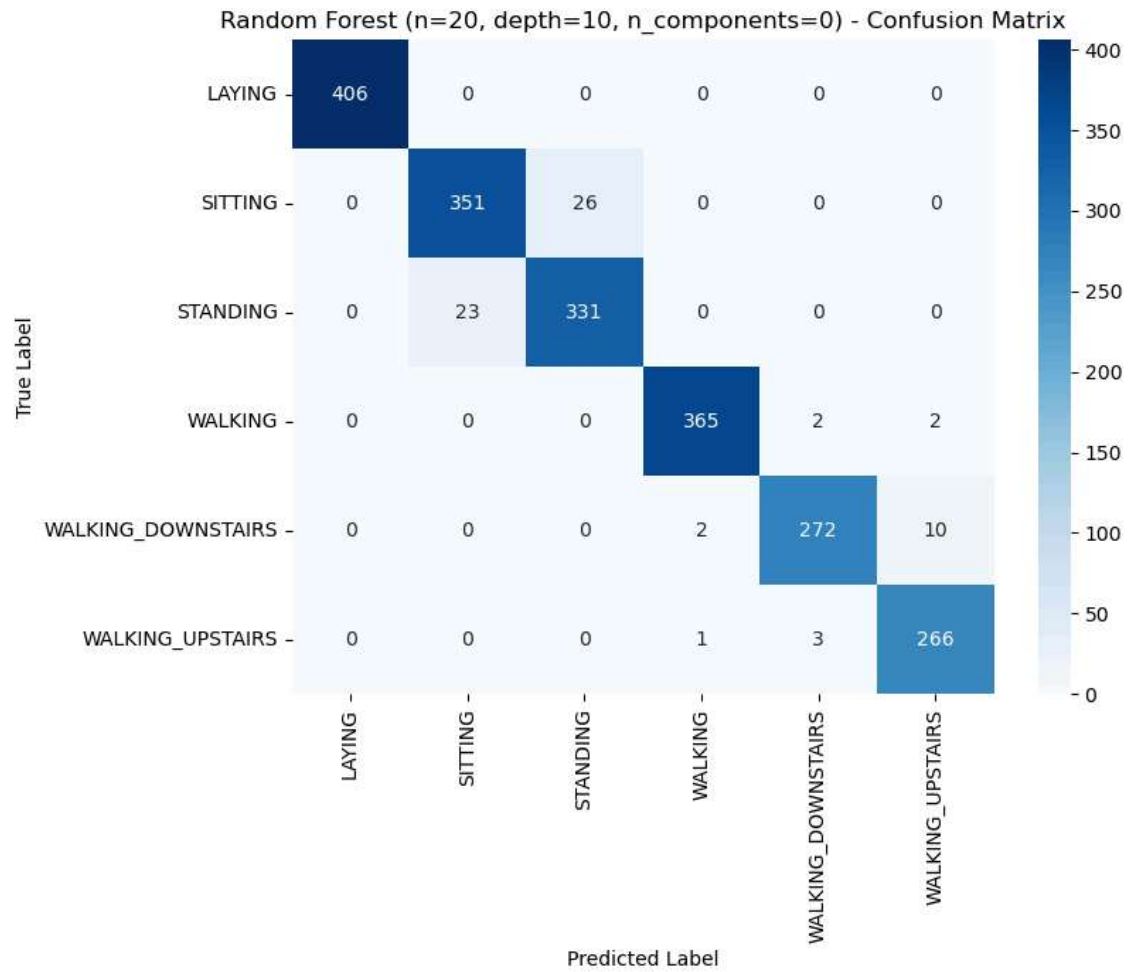


Figure 20: Random forest model confusion matrix when training on raw data

Table 7: Evaluation metrics on the random forest model

Accuracy	0.9665
Sensitivity	0.9664
Specificity	0.9933
F1 Score	0.9661

Explained variance ratio: 0.50801854, 0.06237329, 0.02631659, 0.02443171, 0.01905611, 0.01628338, 0.01403286, 0.01221456

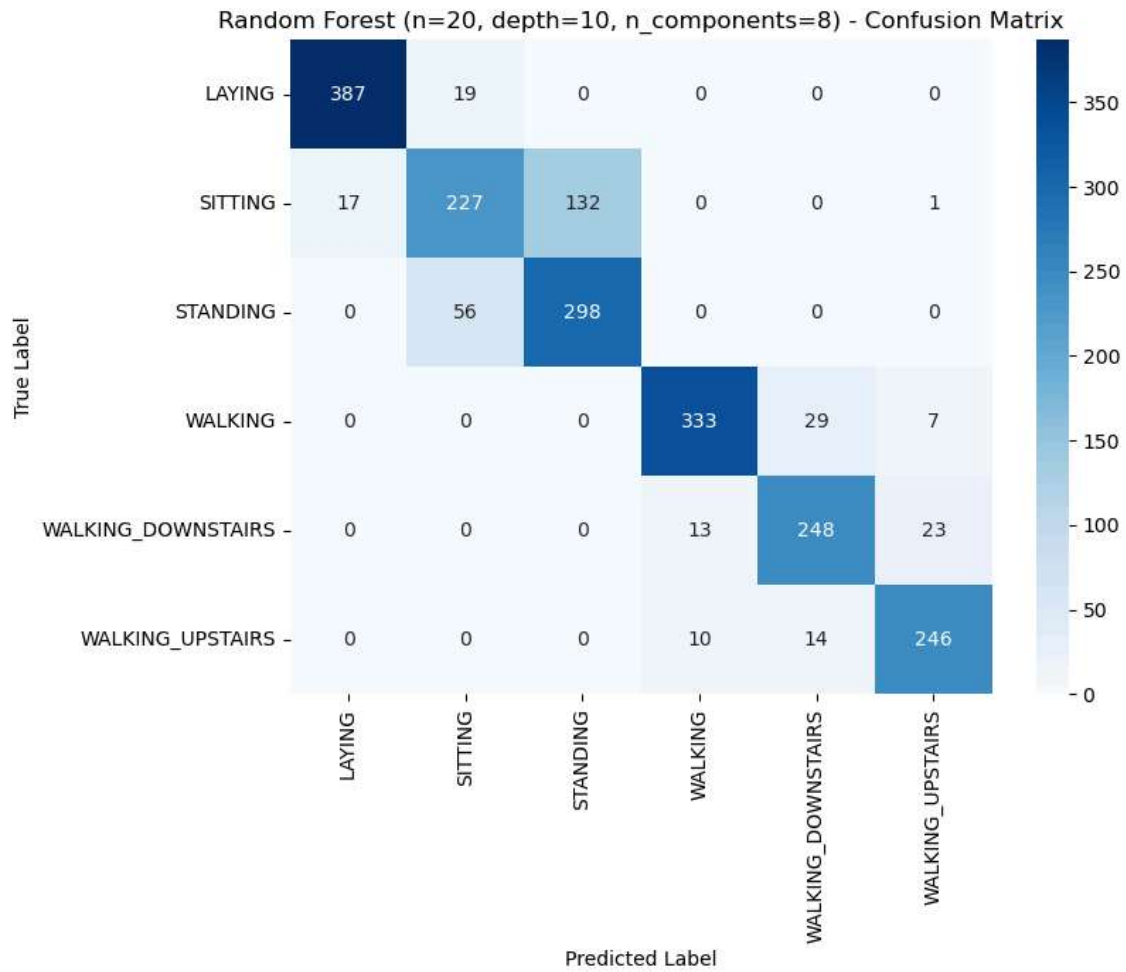


Figure 21: Random forest model confusion matrix when training with PCA of 8 components

Table 8: Evaluation metrics on random forest training with the use of PCA of 8 components

Accuracy	0.8442
Sensitivity	0.8473
Specificity	0.9688
F1 Score	0.8442

Table 9: Evaluation metrics for random forest training with PCA of n components

n_components	Accuracy	Sensitivity	Specificity	F1 Score	Cumulative Explained Variance	Reconstruction Error	Time Taken
1	0.3981	0.402	0.8792	0.399	0.508	0.4902	0.766
2	0.5262	0.5366	0.9046	0.5315	0.5704	0.4281	0.7265

3	0.6762	0.6758	0.9351	0.6694	0.5967	0.4018	0.6013
4	0.783	0.7817	0.9566	0.778	0.6211	0.3775	0.8165
5	0.818	0.815	0.9637	0.812	0.6402	0.3585	0.7961
6	0.8354	0.8358	0.9671	0.8334	0.6565	0.3423	0.8841
7	0.8481	0.8487	0.9696	0.8467	0.6705	0.3283	0.7686
8	0.8602	0.8628	0.9719	0.8609	0.6827	0.3161	0.7677
9	0.8636	0.8674	0.9726	0.866	0.6926	0.3063	1.0468
10	0.8743	0.8777	0.9747	0.8764	0.7022	0.2967	1.3157
11	0.8762	0.8795	0.9751	0.8783	0.7108	0.2882	1.4177
12	0.8825	0.8859	0.9764	0.8845	0.7189	0.2801	1.1946
13	0.8806	0.8841	0.9759	0.8829	0.7267	0.2723	1.3664
14	0.8888	0.8928	0.9776	0.8917	0.7334	0.2656	1.175
15	0.8966	0.8999	0.9792	0.8992	0.7397	0.2593	1.2337
16	0.8942	0.8977	0.9787	0.8968	0.746	0.2531	1.4466

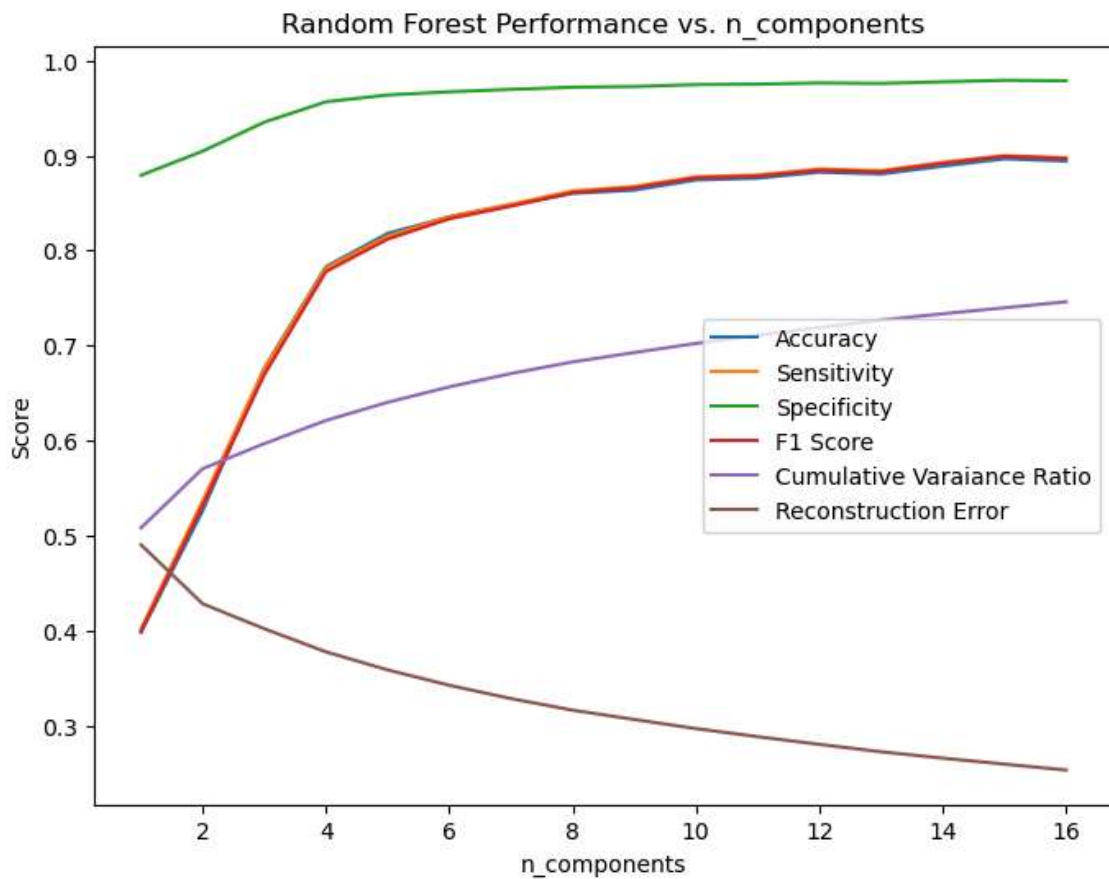


Figure 22: Random forest model's performance metrics versus PCA of n components

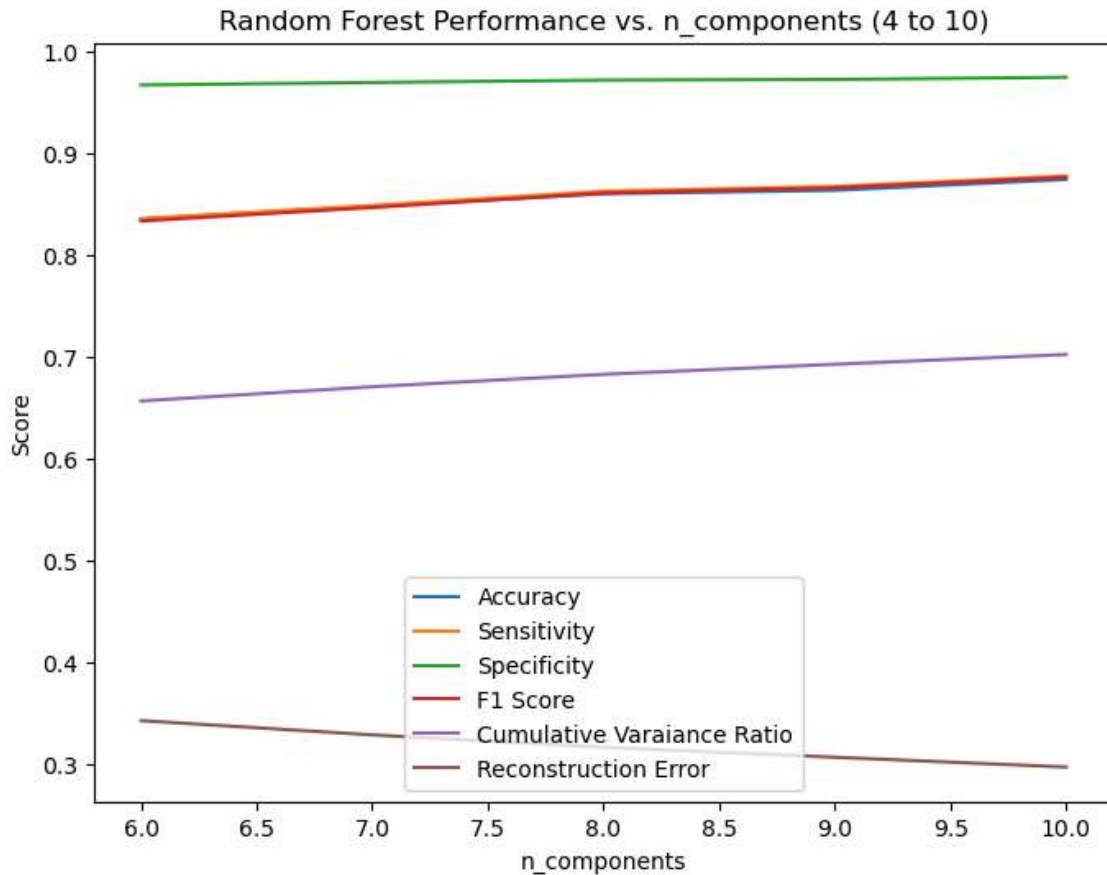


Figure 23: More detailed inspection of optimal components within the range of 1 to 16 components

The criteria for evaluating the most optimal principle component number for number includes:

- Accuracy: The capability to classify the true positive and false negative of each case in the classification dataset. The optimal n component for PCA is found when the accuracy gain starts to diminish (elbow on the curve)
- Cumulative variance ratio: Few components capture the most variance, but not necessarily all variance of the dataset
- Reconstruction error: Preserved data may be important for later. The smaller the error, the better the data preservation.
- Computation time: the shorter the time, the better the performance

Based on these criteria, the acceptable range can be narrowed down to 6 to 10 because of the plateau gain in accuracy, along with sensitivity, specificity and F1 score. The cumulative variance ratio experiences a marginal gain in this interval. Reconstruction error also has a diminishing improvement. Computation time only increases from about 0.9 to 1.1 seconds. The optimal number of components can be 8 to balance between the accuracy and computation time, while other metrics are neglectable because of small changes.

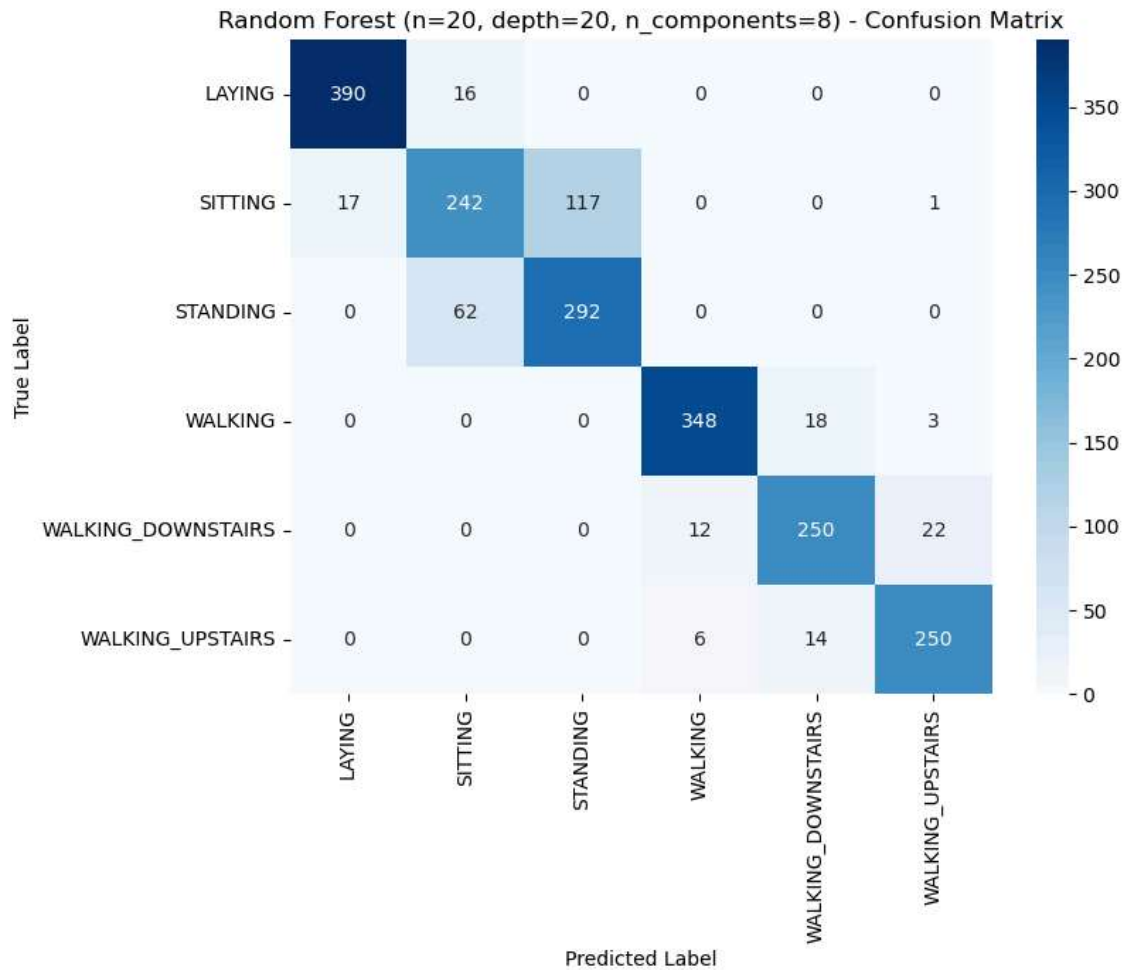


Figure 24: Optimal components for the random forest training

Table 10: Evaluation metrics on random forest training with the use of PCA of 8 components

Accuracy	0.8553
Sensitivity	0.8577
Specificity	0.9709
F1 Score	0.8565

Straight random forest classification with 10 estimators and max depth of 10 metrics:

- Accuracy: 0.9665
- Sensitivity: 0.9664
- Specificity: 0.9933
- F1 Score: 0.9661
- Number of important features (larger than 1%): 34
- Number of important features (larger than 0.1%): 123
- Total number of features: 561

Random forest classification with 20 estimators and max depth of 20, after using PCA for dimensionality reduction of 8 components:

- Cumulative Variance Ratio: 0.6827
- Reconstruction Error: 0.3161
- Accuracy: 0.8641
- Sensitivity: 0.8670
- Specificity: 0.9727
- F1 Score: 0.8658
- Time Taken: 0.9209 seconds
- Number of important features (larger than 1%): 8
- Number of important features (larger than 0.1%): 8
- Total number of features: 8

Based on the metrics, the straightforward random forest classification has better classification performance and better than random forest classification model after dimensionality reduction. The dimension reduction in this analysis may have harmed the performance as the model experiences information loss and lacks useful non-linear features to determine the output. The random forest model is designed to work in a high, high-dimensional dataset. Plus, 34 features have more than 1% of the model's decision-making, which means they are truly important features. In this scenario, feature selection can be a better approach as it matches the common goals of dimension reduction to remove redundant features and reduce computation time. Based on the metrics provided, the straightforward random forest model can perform better than the combination of the PCA and random forest model.